

背包問題：三種另類解說

看不懂 DP 表格怎麼填？沒關係，這裡用三種**完全不同**的角度重新解釋 0/1 背包。挑一個你最喜歡的角度，回頭再看程式碼就會豁然開朗。

題目回顧：4 件物品（重量 `[2,3,4,5]`、價值 `[3,4,5,6]`），背包容量 5，每件最多拿一次。

方法一：超市購物袋類比（最生活化）

想像這個情境

你推著購物車逛超市，背包最多只能裝 5 公斤。貨架上有 4 樣商品：

商品	重量	價值	CP值（價值/重量）
泡麵	2 kg	3 元	1.5
飲料	3 kg	4 元	1.33
書	4 kg	5 元	1.25
籃球	5 kg	6 元	1.2

你的問題：只能裝 5 公斤，怎麼選才能拿最多錢？

先看「直覺會怎麼想」

直覺通常會想：「拿 CP 值最高的！」

泡麵 (CP=1.5) 先拿 → 剩下 3kg
飲料 (CP=1.33) 再拿 → 剩下 0kg
總價值 = 3 + 4 = 7 元 ✓ 剛好是答案！

但這個直覺**不一定對**！換個例子你就知道了：

商品	重量	價值
A	3 kg	100 元
B	2 kg	99 元
C	2 kg	99 元

容量 4kg，CP 值最高的是 A ($100/3 \approx 33$)。

- 貪婪選 A → 剩 1kg → 總價值 100
- 但選 B+C → 4kg 全用 → 總價值 198！

結論：背包問題不能用貪婪，必須考慮「每一種組合」。

那 DP 在幹嘛？

DP 不是在「一個一個選」，而是回答一個問題：

「如果你已經想好前面 i 件東西怎麼選了，現在容量只剩 w ，你能拿多少錢？」

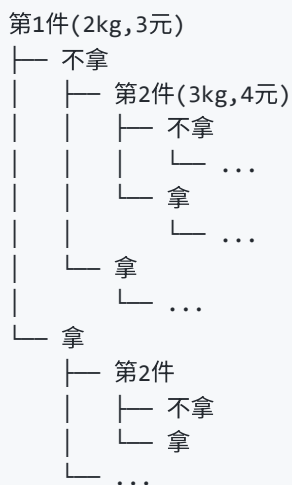
當你看到第 i 件商品時，只有兩個選擇：

1. 不拿 → 答案跟你「沒看到這件商品」時一樣
2. 拿 → 空出它的重量，剩下空間繼續拿前面的

方法二：決策樹類比（最直覺）

每件物品都是「拿 / 不拿」的分岔路

4 件物品，每件有 2 個選擇 → 全部組合有 $2^4 = 16$ 種。暴力法就是把 16 種全列出來，挑不超重且價值最大的。



暴力法為什麼爆炸？

16 種組合對 4 件還行，但 100 件物品 → $2^{100} \approx 1.27 \times 10^{30}$ 種組合，宇宙毀滅都算不完。

但注意一個關鍵事實：很多組合「重複遇到相同的狀態」。

相同狀態的重複

假設你走到某一步，發現：

手上狀態 = 「前 2 件已經決定，剩 3 kg」

這個狀態會出現好幾次！

- 路徑 A：拿了泡麵(2kg)，沒拿飲料 → 剩下 3kg
- 路徑 B：沒拿泡麵，拿了飲料(3kg)...等等，飲料 3kg 直接塞滿 → 剩 0kg，不對

好吧，換個例子想：

「前 2 件已決定、剩 3kg」這個狀態
可以由「前1件已決定」的不同路徑到達

路徑1：第1件不拿 → 剩 5kg → 第2件不拿 → 剩 5kg（不是3，不對）

沒關係，重點是這個概念：

同一種「剩餘容量」狀態，不管你是怎麼走到的，
之後的選擇都一樣 → 只要記一次答案就好！

DP 的價值：把「狀態」從「完整路徑」變成「(處理到第幾件, 剩多少容量)」，大量路徑會合併成同一個狀態，重複計算全部消除。

決策樹 vs DP 的對照

項目	決策樹（暴力）	DP 填表
狀態	整條路徑	(i, w)：處理到第 i 件、剩 w 容量
狀態數	2^N 種	$N \times W$ 種
重複計算	有	無
範例 $N=100$	10^{30}	$100 \times 10000 = 10^6$

狀態從「路徑」簡化成「位置+容量」，就是 DP 的一切。

方法三：遞迴思考類比（最數學）

先寫遞迴，再翻譯成迴圈

0/1 背包的本質可以用一個遞迴函數描述：

$f(i, w)$ = 前 i 件物品、容量 w 的最大價值

```
f(i, w) = max(
    f(i-1, w),                ← 不拿第 i 件
    f(i-1, w - weight[i]) + value[i] ← 拿第 i 件（容量夠的話）
)
```

邊界： $f(0, w) = 0$ （沒有物品可拿）

用遞迴展開小例子

物品：A(2kg,3元)、B(3kg,4元)，容量 5。

```
f(2, 5) = max(  
    不拿B: f(1, 5),  
    拿B:   f(1, 2) + 4  
)  
  
f(1, 5) = max(  
    不拿A: f(0, 5) = 0,  
    拿A:   f(0, 3) + 3 = 0 + 3 = 3  
) = 3  
  
f(1, 2) = max(  
    不拿A: f(0, 2) = 0,  
    拿A:   f(0, 0) + 3 = 3  
) = 3  
  
所以 f(2, 5) = max(3, 3+4) = 7 ✓
```

觀察遞迴中「重疊的子問題」

展開 $f(2, 5)$ 時， $f(1, 2)$ 被呼叫。如果繼續展開 $f(3, 5)$ ， $f(1, 2)$ 可能又會被呼叫一次！

```
f(3, 5)  
├─ 不拿C: f(2, 5) → 裡面又會算 f(1, 2) ← 重複！  
└─ 拿C:   f(2, 1) → 裡面又會算 f(1, 1)
```

重疊子問題 = 同一個 $f(i, w)$ 被重複求解。

把遞迴「反過來」變成迴圈

遞迴是從上往下（先問大的，再拆小的）。DP 迴圈是從下往上（先算小的，再組大的）：

遞迴 Top-Down：
 $f(2,5) \rightarrow f(1,5) \rightarrow f(0,5)$ （最小，回傳 0）→ 一路組回去

DP Bottom-Up：
i=0: 算完所有 $f(0, *) = 0$
i=1: 用 $f(0, *)$ 算 $f(1, *)$
i=2: 用 $f(1, *)$ 算 $f(2, *)$
...

因為只會用到「上一層」的結果，一維陣列就夠了！

為什麼空間壓縮後要倒序？

一維陣列 $dp[w]$ 代表「目前為止的最優解」。處理第 i 件時， $dp[w]$ 應該用還沒處理第 i 件的舊值計算。

正序 (w 小→大) :
dp[2] 先更新 (含了物品 A)
dp[4] 再更新時用 dp[2] → 已經含 A → A 被用兩次! X

倒序 (w 大→小) :
dp[4] 先更新, 用的 dp[2] 是舊值 (不含 A)
dp[2] 才更新
→ A 最多被用一次 ✓

遞迴、暴力、DP 三者對照

方法	作法	時間複雜度	記憶體
暴力窮舉	列出所有 2^N 種組合	$O(2^N)$	$O(1)$
純遞迴	每次都重算子問題	$O(2^N)$	$O(N)$ (堆疊)
遞迴+記憶化	記錄算過的 $f(i,w)$	$O(N \times W)$	$O(N \times W)$
DP 填表	由小到大填表格	$O(N \times W)$	$O(W)$ (壓縮)

三種方法的共通核心

不管你用哪種角度理解，重點都是：

1. 找到「選或不選」這個二元決策

面對第 i 件物品，只有兩條路：
拿 → 付出重量，得到價值
不拿 → 什麼都不變

2. 狀態 = (處理到第幾件, 剩多少容量)

把 2^N 種路徑，收斂成 $N \times W$ 種狀態。

3. 填表的順序要確保「依賴的狀態先算好」

表格版本	依賴關係	保證方式
2D 版	$dp[i][w]$ 依賴 $dp[i-1][*]$	外層迴圈 i 遞增
1D 版	$dp[w]$ 依賴「舊」 $dp[\text{小容量}]$	內層迴圈 w 倒序

回到程式碼：現在你應該看得懂了

```
def knapsack(weights, values, W):
    dp = [0] * (W + 1)
    for i in range(len(weights)):          # 逐一處理每件物品
        for w in range(W, weights[i] - 1, -1): # 容量從大到小（倒序！）
            dp[w] = max(dp[w],              # 不拿第 i 件
                        dp[w - weights[i]] + values[i]) # 拿第 i 件
    return dp[W]
```

程式碼	對應的概念
for i in range(len(weights))	走過每一件商品，做「拿/不拿」的決策
for w in range(W, ...) 倒序	確保 dp[小容量] 還是「不含本件」的舊值
dp[w] = max(dp[w], ...)	拿 vs 不拿，取價值高的
return dp[W]	容量 W 能拿到的最大價值

立刻驗證你懂了沒

不查解答，試著回答：

#	問題	答案
1	為什麼貪婪（拿 CP 值最高）不一定對？	反例：容量 4，(3kg,100元) vs (2kg,99元)+(2kg,99元)，後者 198>100
2	DP 比暴力快在哪裡？	暴力有 2^N 條路徑，DP 只有 N×W 種狀態
3	為什麼倒序更新？	避免用「已含本件」的新值，造成同一件被重複選
4	如果改成無限背包，迴圈怎麼改？	正序 for w in range(weights[i], W+1)